FRONT-END CODE DOCUMENTATION REPORT

Technical Codebase Architecture & Developer Setup Specification

TECHNICAL DELIVERABLE

Project Name	*[Insert Application Name]*	**Tech Stack**	HTML5, CSS3, JavaScript (ES6+), REST API
Student Developer	*[Name & Register Number]*	**Repository URL**	*[https://github.com/org/repo-name]*
Academic Course	25CS208 – Web Technology	**Deployment URL**	*[https://project.vercel.app]*

1. REPOSITORY DIRECTORY STRUCTURE & MODULAR ORGANIZATION

Folder Structure Tree

```
root/
├── index.html           # Main semantic HTML5 markup
├── css/
│   ├── variables.css    # Design tokens & dark mode rules
│   └── styles.css       # Layout grids, flexbox & animations
├── js/
│   ├── app.js           # Main application entry & state
│   ├── api.js           # REST API fetch module & handlers
│   └── validation.js    # Client-side form regex validators
└── assets/              # WebP images & SVG vector icons
```

Architectural Principles

- **Separation of Concerns:** Markup, styling tokens, API handling, and UI dynamic scripts are segregated into distinct modules.
- **CSS Custom Variables:** Palette colors, spacing scale, and dark/light themes declared globally in `variables.css`.
- **Zero Build Dependency:** Standard ES6 JavaScript modules usable directly natively in modern web browsers.

2. CORE COMPONENT MODULES & TECHNICAL LOGIC

Module / File	Core Functions / Event Listeners	Technical Logic & State Management
`js/app.js`	`initApp()`, `toggleTheme()`, `renderCards()`	Manages application state, listens to DOM click events, toggles dark mode, updates UI dynamically without page refresh.
`js/api.js`	`fetchData(endpoint)`, `handleAPIError()`	Executes asynchronous HTTP `GET/POST` requests via `fetch()` API with `try...catch` defensive error handling.
`js/validation.js`	`validateForm(e)`, `checkRegexPattern()`	Validates client form inputs on blur/submit events using regular expressions; displays inline visual error states.
`js/storage.js`	`saveLocal(key, val)`, `loadLocal(key)`	Persists user settings, dark mode preferences, and offline message history inside `window.localStorage`.

3. CODE QUALITY, ACCESSIBILITY (A11Y) & SECURITY MEASURES

Semantic HTML5	Uses strict structural tags (`<header>`, `<nav>`, `<main>`, `<article>`, `<footer>`) to ensure clean SEO and document outline.
WCAG 2.1 Accessibility	Includes `aria-expanded` and `aria-label` tags on interactive controls; enforces `:focus-visible` indicators for keyboard navigation.
Security (AppSec)	Sanitizes user input text strings using DOM text nodes to prevent Cross-Site Scripting (XSS); API keys stored in client `.env` configurations.

4. DEVELOPER SETUP & DEPLOYMENT INSTRUCTIONS

Local Development Setup:

1. Clone the repository: `git clone https://github.com/org/repo-name.git`
2. Navigate into project directory: `cd repo-name`
3. Launch using any standard HTTP local server (e.g., VS Code Live Server or `npx serve`).

Continuous Deployment Pipeline:
Connected directly to Vercel/Netlify cloud. Commits pushed to the `main` branch automatically trigger production builds with SSL encryption enabled.

5. TECHNICAL EVALUATION SIGN-OFF

Student Lead Developer:	**Faculty / Industry Reviewer:**
Signature: _____	Signature: _____
Name: *[Student Name]* | Date: _____	Name: *[Faculty Mentor]* | Date: _____